Lab10 Ensemble Learning

Jupyter Notebook — original cells shown separately and in order

CELL 01 | CODE | EXECUTION COUNT: 1

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

CELL 02 | CODE | EXECUTION COUNT: 2

```
df = read.csv("")
print("Dataset loaded successfully")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
```

OUTPUT

```
Dataset loaded successfully
Number of samples: 569
Number of features: 30
```

CELL 03 | CODE | EXECUTION COUNT: 3

```
df = pd.DataFrame(X, columns=data.feature_names)
df["Target"] = y
df.head()
```

OUTPUT

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  Target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
```

CELL 04 | CODE | EXECUTION COUNT: 4

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Training set size:", X_train.shape[0])
print("Testing set size:", X_test.shape[0])
```

OUTPUT

```
Training set size: 455
Testing set size: 114
```

CELL 05 | CODE | EXECUTION COUNT: 5

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
lr=LogisticRegression()
lr.fit(X_train,y_train)
lr.prediction=lr.predict(X_test)
lr_accuracy=accuracy_score(y_test,lr.prediction)
print("Logistic Regression Accuracy:", lr_accuracy)
```

OUTPUT

```
Logistic Regression Accuracy: 0.9736842105263158
```

CELL 06 | CODE | EXECUTION COUNT: 6

```
rf=RandomForestClassifier()
rf.fit(X_train,y_train)
rf.prediction=rf.predict(X_test)
rf_accuracy=accuracy_score(y_test,rf.prediction)
print("Random Forest Accuracy:", rf_accuracy)
```

OUTPUT

```
Random Forest Accuracy: 0.9649122807017544
```

CELL 07 | CODE | EXECUTION COUNT: 7

```
results=pd.DataFrame({"Model":["Logistic Regression","Random
Forest"],"Accuracy":[lr_accuracy,rf_accuracy]})
results
```

OUTPUT

```
                Model  Accuracy
0  Logistic Regression  0.973684
1        Random Forest  0.964912
```

CELL 08 | CODE | EXECUTION COUNT: 8

```
plt.figure(figsize=(8,6))
plt.bar(results["Model"],results["Accuracy"])
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")
plt.show()
```

OUTPUT

```
<Figure size 800x600 with 1 Axes>
```

CELL 09 | CODE | EXECUTION COUNT: 9

```
print("logistic regression:")
print(classification_report(y_test,lr.prediction))
print("Random Forest:")
print(classification_report(y_test,rf.prediction))
#
```

OUTPUT

```
logistic regression:
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        43
           1       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114

Random Forest:
              precision    recall  f1-score   support

           0       0.98      0.93      0.95        43
           1       0.96      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```

CELL 10 | CODE | EXECUTION COUNT: 10

```
print("performance comparision")
print("logistic regression accuracy:", round(lr_accuracy,2))
print("Random Forest accuracy:", round(rf_accuracy,2))
```

OUTPUT

```
performance comparision
logistic regression accuracy: 0.97
Random Forest accuracy: 0.96
```

CELL 11 | CODE

(Empty cell)

CELL 12 | CODE

(Empty cell)

CELL 13 | CODE

(Empty cell)